Universidad Nacional Autónoma de México

Computer Engineering

COMPILERS

PARSER & SDT

STUDENTS:
321117744
321339009
321008215
321032308
320237012

Group: 5
Team: 4
Semester: 2026-II

México, CDMX. April 30th, 2026

Contents

1 Introduction **2**
1.1 Problem Statement 2
1.2 Motivation 2
1.3 Objective 2

2 Theoretical Framework **2**

3 Grammar Definition **3**
3.1 Program Structure 3
3.2 Instructions 3
3.3 Statements 4
3.4 Control Structures 4
3.5 Expressions 4

4 System Design **4**
4.1 Lexical Analyzer 4
4.2 Parser 5
4.3 Semantic Analyzer 6

5 Results **7**
5.1 Correct Program 7
5.2 Syntax Error 8
5.3 Semantic Error 9

6 Conclusions **11**

1 Introduction

1.1 Problem Statement

This project extends the previously developed lexical analyzer by implementing a parser and a Syntax-Directed Translation (SDT) system based on a Context-Free Grammar.

1.2 Motivation

To apply the concepts learned in class related to parsing techniques and semantic analysis, and to understand how different phases of a compiler interact.

1.3 Objective

To develop a syntactic analyzer capable of validating program structure, generating an Abstract Syntax Tree (AST), and performing semantic verification using SDT rules.

2 Theoretical Framework

A programming language is structured through syntax and semantics.

- **Syntax:** Syntax studies the way in which words are combined and what the relationships between them are to form an idea. It helps us to establish the rules that give order to a language. Also, to avoid misunderstandings and to give consistent messages [1].
- **Semantics:** Semantics is the one that studies the words, signs, and the meaning of each of them [2]. [3]

Understanding language is important for the compiler, which is defined as a computer program that converts code from one programming language, to another programming language. The compiler transforms high-level language into low-level language, and it does this through a process that has the next phases:

1. Lexical analysis
2. Syntax analysis
3. Semantic analysis
4. Optimization.
5. Code generation [4].

The lexer, a previous project, did the first analysis (lexical). The present project adds the Parser, which consists of syntax analysis, and the SDT, which extends the analysis by including semantic actions.

A **parser** is the syntactic analysis where the computer interprets the grammatical structure of a text in language, whether it is a natural or a programming language.

The **parser** breaks down words and symbols into an organized hierarchy (like a syntax tree), to identify how the elements of a sentence relate to each other according to rules established in a pre-established grammar. Depending on the order of analysis, it can be ascending (**bottom-up**) or descending (**top-down**) [5].

SDT, also known as "**Syntax Directed Translation**", is a technique where semantic actions are included alongside the grammar production. Allows the execution of semantic actions associated with grammar rules, enabling type checking and symbol management. [6]

The **AST** represents the logical structure of the program.

3 Grammar Definition

The following Context-Free Grammar defines the structure of the Juan Pato language.

3.1 Program Structure

```
program     Start { instructions }

instructions     instruction instructions
             | instruction
```

3.2 Instructions

```
instruction     writing
            | reading
            | declaration
            | assignment
            | if
            | if-else
            | while
            | for
            | comment
```

3.3 Statements

```
writing     printf expression ;
reading     read ID ;

declaration     TYPE ID ;
declaration     TYPE ID = expression ;

assignment     ID = expression ;
```

3.4 Control Structures

```
if     if expression then { instructions }

if-else     if expression then { instructions }
          else { instructions }

while     while expression { instructions }

for     for ID from expression to expression { instructions }
```

3.5 Expressions

```
expression     expression ARIT_OP expression
           | expression COMP_OP expression
           | expression BOOL_OP expression
           | ( expression )
           | ID
           | INTEGER
           | REAL
           | STRING
           | BOOL
```

4 System Design

The compiler consists of three main modules.

4.1 Lexical Analyzer

The project's lexical analyzer aims to transform the input, initially represented as text, into a sequence of tokens using regular expressions, in order to prepare the information for the compiler's subsequent stages.

One of the libraries for implementing lexical analysis is RPLY. This is a pure Python parser generator that offers easy-to-use tools to indicate the rules needed for this type of analysis through a LexerGenerator instance, which represents a set of rules that match pieces of text that should either be turned into tokens or ignored by the Lexer. Rules are added using the method called add or ignored with 'ignore'.

The file called 'AnalizadorLexico.py' is the one that carries the lexical analysis of the file, the first stage of the full compilation process. It employs the class lexer from the lexer generator library, this allows entries to be added to the lexer, by which we can define tokens with a name to identify each of them by, as well as the regular expression that the program expects to find in the .jp files and that it uses for cataloging each part of its contents. In doing so, we can define the grammar of the language for the compiler, and detect if a supposedly correct file follows the grammar properly or if it has lexical errors. It is able to classify all of the symbols and at the end of its execution, redirect the output to the main.py file to present the information to the user.

Some of the tokens that it is designed to accept are: the start of a program, the brackets that limit its span, the type of a variable, a variable name, a number, an operator (arithmetic, boolean, assignation), as well as what to ignore like white spaces and comments.

4.2 Parser

The parser for this project was designed to validate the syntax, build an Abstract Syntax Tree (AST), and generate a symbol table from the input program. The parser is bottom up (LALR).

As a first step, it verifies that the tokens have the correct structure. If there is a mistake, the parser will report us the syntax error with its line.

After validating the syntax, the parser builds the Abstract Syntax Tree (AST). This tree represents in a graphical and in a hierarchical way, the structure of the program. Each node in the tree is an instruction, expression, or control structure (such as if, while, or for). This helps the program to be easier to analyze.

Also, the parser creates a symbol table. This table shows us information about the variables used in the program, such as their type, and value. It also helps to detect cases where a variable is used without being declared.

There were several tests performed using different input programs.

One of the programs had variable declarations and arithmetic operations. When it was analyzed,the parser validated the syntax, built the AST, and stored the variables in the symbol table.

Other programs with control structures like **if, else**, **while**, and **for** were tested too. The parser recognized these structures and represented them correctly in the AST.

But in error cases where there were syntax errors or undeclared variables, the parser reported errors or warnings. This shows that the validation worked correctly.

Altogether, the tests showed that the parser correctly validates syntax, builds the AST, and generates the symbol table. And, when it's necessary, tell us where there is a mistake.

4.3 Semantic Analyzer

The semantic analysis represents the phase where the compiler validates the logical coherence of the program. It is not enough for the code to comply with grammatical rules; the compiler functions as a filter that ensures only "perfect logic" progresses toward execution

For this purpose, the implementation of the SemAnalyzer class receives the Abstract Syntax Tree (AST) and traverses it recursively. In this process, each node of the tree is examined to guarantee that operations between variables and values are compatible, preventing underlying inconsistencies from becoming critical failures during runtime.

The operation of this analyzer depends on the Symbol Table (sym_table), which acts as the central memory of the system by storing identifiers and their data types, whether they are integers, reals, or booleans. During the AST traversal, the tipoSubTree method consults this table every time a variable is referenced. This step is fundamental, as without such context, the analyzer has no means to verify if a variable has been previously declared or if the assigned data type corresponds to the nature of the operation. In this way, the symbol table provides real meaning to the hierarchical structure of the tree.

It is imperative to distinguish this process from syntactic analysis. While a syntax error is a failure of form, such as the omission of a semicolon that prevents the construction of the tree, a semantic error arises when the structure is valid but the instruction is logically impossible.

The analyzer is designed to detect three main categories of failures: type mismatch (such as assigning a text string to an integer variable), the use of undeclared variables, and the presence of non-boolean conditions in control structures. Detecting these failures significantly increases the robustness of the source code.

Finally, the integrity of the validation is managed through the num_errors variable. If, upon concluding the tree review, the counter shows a value greater than zero, the compilation process is interrupted. This mechanism ensures that the user

receives clear feedback on logical errors before proceeding with the execution of the software. In conclusion, the semantic analyzer verifies the coherence of the program, elevating the role of the compiler from a simple translator to a quality control tool that only allows the flow of semantically impeccable code.

5 Results

5.1 Correct Program

Parsing succeeds, AST is generated, and SDT is verified.

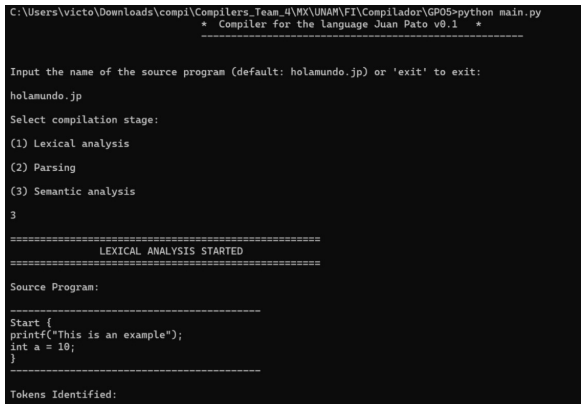
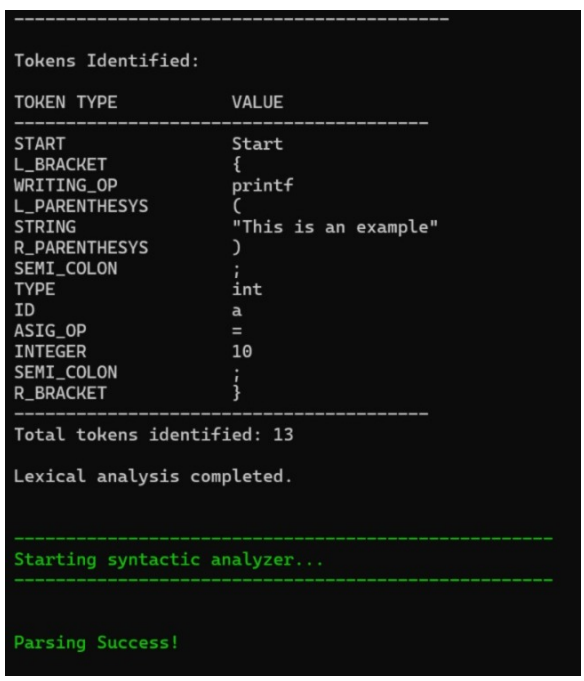
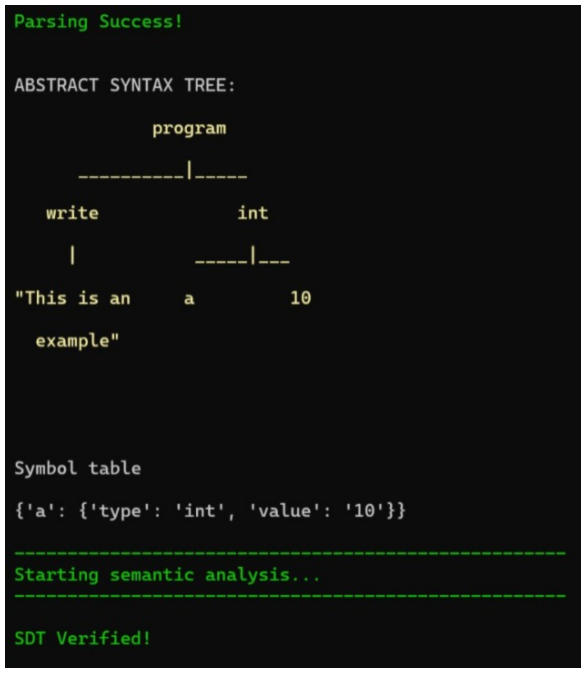


Figure 1: Analysis on a flawless file - holamundo.jp





5.2 Syntax Error

Parsing fails and semantic analysis is not executed.

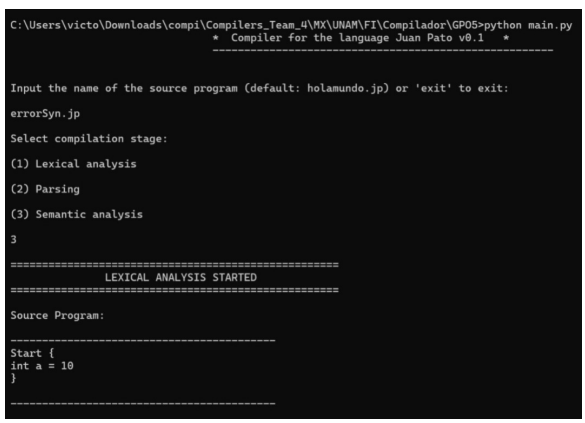


Figure 2: Showcasing a program with syntax errors - errorSyn.jp

```
----------------------------------------

Tokens Identified:

TOKEN TYPE          VALUE
----------------------------------------
START               Start
L_BRACKET           {
TYPE                int
ID                  a
ASIG_OP             =
INTEGER             10
R_BRACKET           }
----------------------------------------
Total tokens identified: 7

Lexical analysis completed.


----------------------------------------------------
Starting syntactic analyzer...
----------------------------------------------------

>> SYNTAX ERROR at token: Token('R_BRACKET', '}')
   Line: 3 Column: 1

Parsing error...
```

5.3 Semantic Error

Parsing succeeds but SDT reports an error.

```
C:\Users\victo\Downloads\compi\Compilers_Team_4\MX\UNAM\FI\Compilador\GPO5>python main.py
                                  *  Compiler for the language Juan Pato v0.1   *
                                  ----------------------------------------------


Input the name of the source program (default: holamundo.jp) or 'exit' to exit:

errorSem.jp

Select compilation stage:

(1) Lexical analysis

(2) Parsing

(3) Semantic analysis

3

==================================================
              LEXICAL ANALYSIS STARTED
==================================================

Source Program:

----------------------------------------
Start {
    # 1. Type Mismatch (The most common semantic error)
    int edad = 25;
    edad = "veinticinco"; # Error: Variable 'int' receiving a 'string'

    # 2. Undeclared Variable (Caught by the Symbol Table check)
    puntos = 100; # Error: 'puntos' has not been declared
```

Figure 3: Showcasing a program with semantic errors - errorSem.jp

```
    # 3. Invalid Logic Condition (caught by 'if' check)
    real pi = 3.1416;
    if pi then {
        printf "Esto no deberia funcionar";
    } # Error: Expected 'bool' for 'if' condition, got 'real'
}
----------------------------------------

Tokens Identified:

TOKEN TYPE          VALUE
----------------------------------------
START               Start
L_BRACKET           {
COMMENT             # 1. Type Mismatch (The most common semantic error)
TYPE                int
ID                  edad
ASIG_OP             =
INTEGER             25
SEMI_COLON          ;
ID                  edad
ASIG_OP             =
STRING              "veinticinco"
SEMI_COLON          ;
COMMENT             # Error: Variable 'int' receiving a 'string'
COMMENT             # 2. Undeclared Variable (Caught by the Symbol Table check)
ID                  puntos
ASIG_OP             =
INTEGER             100
SEMI_COLON          ;
COMMENT             # Error: 'puntos' has not been declared
COMMENT             # 3. Invalid Logic Condition (caught by 'if' check)
TYPE                real
```

```
TYPE                real
ID                  pi
ASIG_OP             =
REAL_NUMBER         3.1416
SEMI_COLON          ;
IF                  if
ID                  pi
THEN                then
L_BRACKET           {
WRITING_OP          printf
STRING              "Esto no deberia funcionar"
SEMI_COLON          ;
R_BRACKET           }
COMMENT             # Error: Expected 'bool' for 'if' condition, got 'real'
R_BRACKET           }
----------------------------------------
Total tokens identified: 35

Lexical analysis completed.


--------------------------------------------------------
Starting syntactic analyzer...
--------------------------------------------------------

**WARNING: Variable 'puntos' has not been declared. Initialized with None.

Parsing Success!
```

```
ABSTRACT SYNTAX TREE:
                                                                                                                                                                                                                                                program
    ------------------------------------------------------
```

6 Conclusions

An analysis of the results obtained underscores the importance of applying compiler design theory to achieve consistent and reliable source code processing. The integration of lexical, syntactic, and semantic analysis demonstrates how each phase contributes to the correct interpretation of a program: from transforming plain text into structured tokens, through validating its grammatical structure, to ensuring its logical coherence. In this context, the parser and the SDT extend the compiler's functionality by validating syntax and semantics, thus ensuring the correct structure and logic of the program.

The results confirm that the formal definition of grammars, the use of parsing techniques such as LALR, and the implementation of semantic rules using symbol tables and tree traversal are essential for detecting inconsistencies at different levels of abstraction. This layered validation approach allows for the precise identification of errors according to their nature, reflecting the effectiveness of the theoretical foundations applied in practice.

Furthermore, the interaction between the Abstract Syntax Tree and the symbol table is fundamental for assigning meaning to the program beyond its structure, reinforcing the idea that compilation is not only a translation process but also a verification mechanism. Overall, the results demonstrate that the correct application of these concepts allows for the construction of a robust system capable of guaranteeing both syntactic correctness and semantic validity.

References

[1] B. M. Islas, "Sintaxis Elemental," *Unidades de Apoyo para el Aprendizaje (UAPA)*, CUAIEED/FES Acatlán-UNAM, 2018. [Online]. Available: `https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/1723/mod_resource/content/9/contenido/index.html` [Accessed: Apr. 29, 2026].

[2] Universidad Complutense de Madrid, "La semántica," *Plataforma gramatical para la enseñanza del español como lengua extranjera*, n.d. [Online]. Available: `https://www.ucm.es/plataformaele/la-semantica` [Accessed: Apr. 29, 2026].

[3] A. Martínez Romero, J. L. Ortega Sánchez, and J. de J. Alba Romero, "Lenguaje: instrumento del desarrollo humano" (English title: "Language: instrument of human development"), *Revista Digital Universitaria (RDU)*, vol. 22, no. 5, Sep.-Oct. 2021. doi: 10.22201/cuaieed.16076079e.2021.22.5.10.

[4] IBM, "What is a Compiler?," *IBM Think*, n.d. [Online]. Available: `https://www.ibm.com/mx-es/think/topics/compiler` [Accessed: Apr. 29, 2026].

[5] Marco, "Parsing," *MSMK University*, Jul. 1, 2025. [Online]. Available: `https://msmk.university/parsing/` [Accessed: Apr. 29, 2026].

[6] GeeksforGeeks, "Syntax-Directed Translation Schemes," *GeeksforGeeks*, Jul. 23, 2025. [Online]. Available: https://www.geeksforgeeks.org/compiler-design/syntax-directed-translation-schemes/ [Accessed: Apr. 29, 2026].